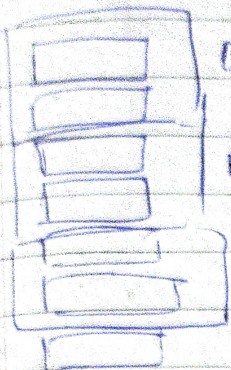


File Structure in DBMS

I. Sorted and Unsorted file structure



Block 1 The data of a table can be stored in the form of blocks contiguously in a memory.

Block 2

Block 3

Now, this data can be stored either in a sorted or unsorted fashion.

Note:

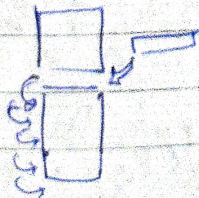
If sorted wrt one specific field (attribute), the other columns will still be unsorted. Usually, that column is sorted which is the most commonly used one (For eg: Student ID).

Adv.

sorted

- Searching a record (using the primary (sorted) attribute) will take $O(\log n)$ time

- Insertion will take linear time



- Deletion will take linear time



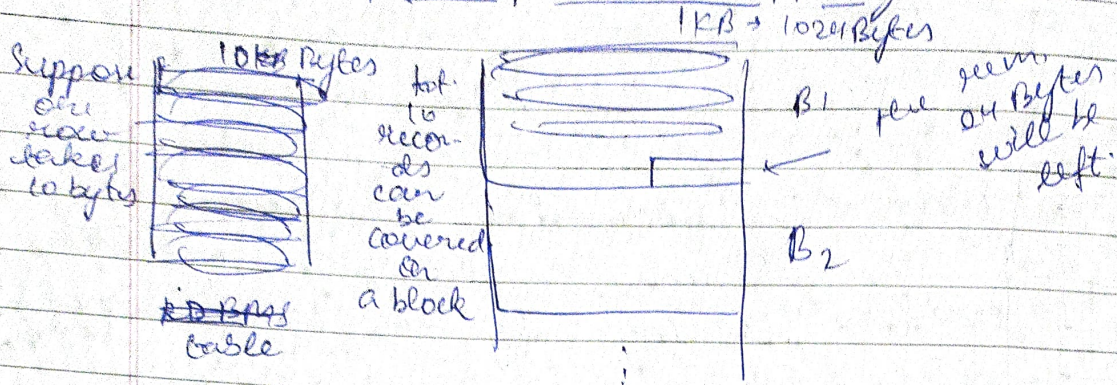
unsorted

- Searching will take linear time.

- Insertion will take only $O(1)$ time as the table is already unsorted.

- Deletion will take linear time to find & delete

II Spanned / Unspanned Mapping



File structure

Spanned Mapping

Those rem. 24 bytes are also utilised and some part of the row is stored in one block and some in some other block.

Problems:- Finding the next rem. part of the row can be difficult to find. 2 blocks will be searched for to find 1 row. Finding a block can take additional time specially if the file structure is not stored synchronously.

Unspanned Mapping

That 24 bytes are left as it is (some amount of memory wastage) if we allow external fragmentation.

Adv:

- 1 row in 1 block

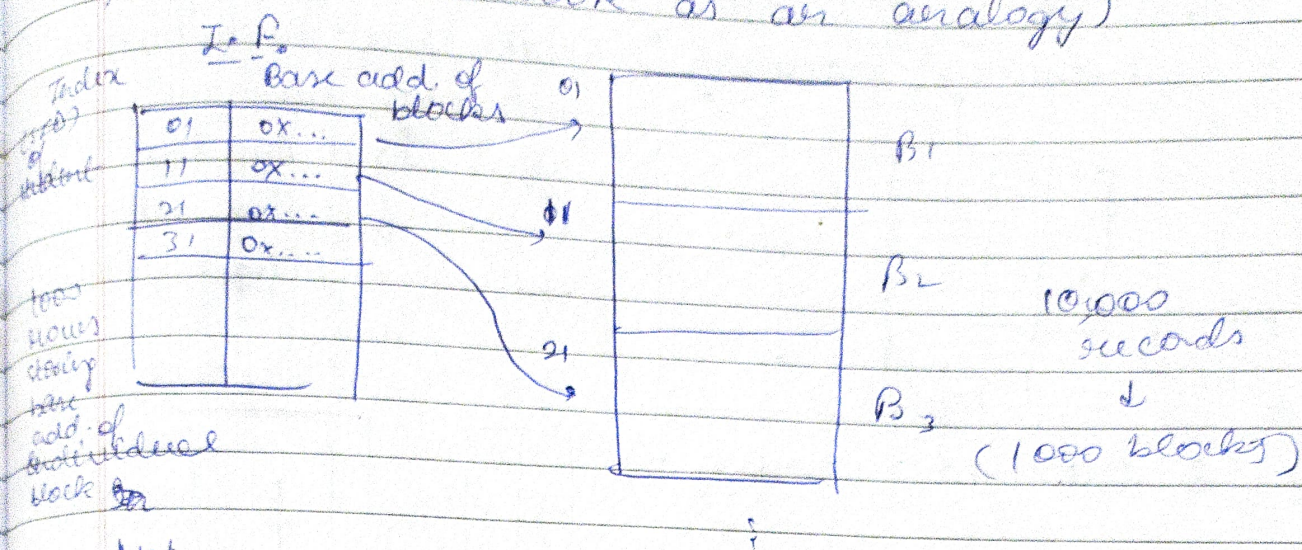
- Only 1 block needs to be searched

Disadv:

- Bit of memory wastage (but that's okay ??)

III Indexing in DBMS

- Involves creation of an index file (similar to a book as an analogy)



Note:

- [Finding the block takes max. time. Finding a particular row in a block takes comparatively negligible time.

If stored in sorted order:

Fetching the req. row in the Index File (I.F.) will take $O(\log_2 1000)$ time

* How is it faster?

more space, more time (Rule of thumb) here,

- 1) No. of cols are less (only 2) as compared to file structure (horizontally less space)
- 2) No. of rows are less (less by 10 times for this case) { spare indexing }

If stored in unsorted order:

No. of rows in I.F. = No. of records in the file structure (10,000)

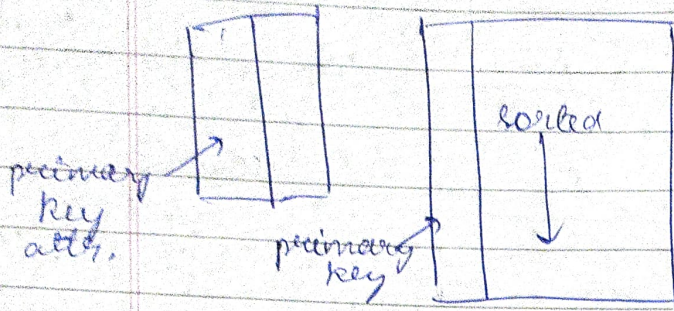
* How is it faster?

- 1) Though, vertically it's the same. Horizontally, it can be proved that accessing records is time

III A. Types of Indexing

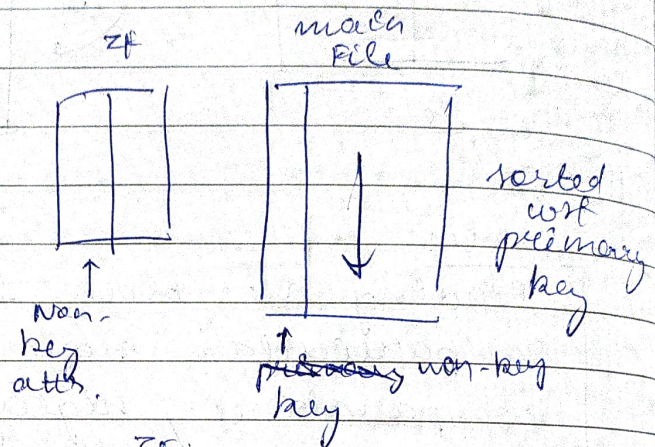
1. Single level indexing

① Primary Indexing -

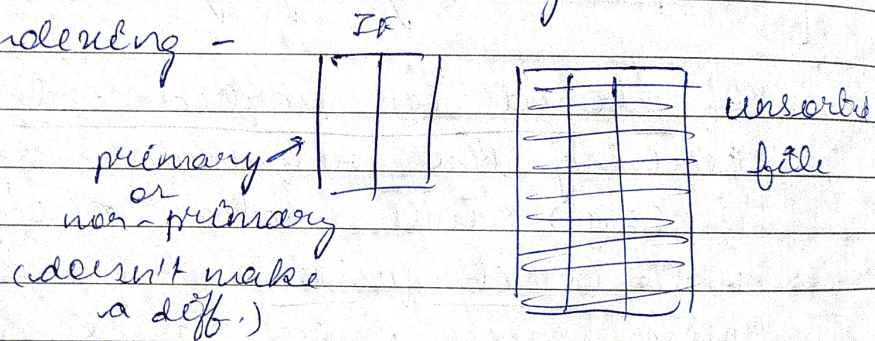


Main file sorted w.r.t primary key and attr. used for the Index file is also the same primary key

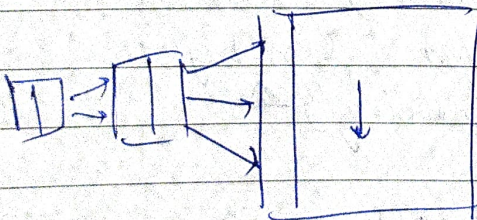
② Clustered Indexing -



③ Secondary Indexing -



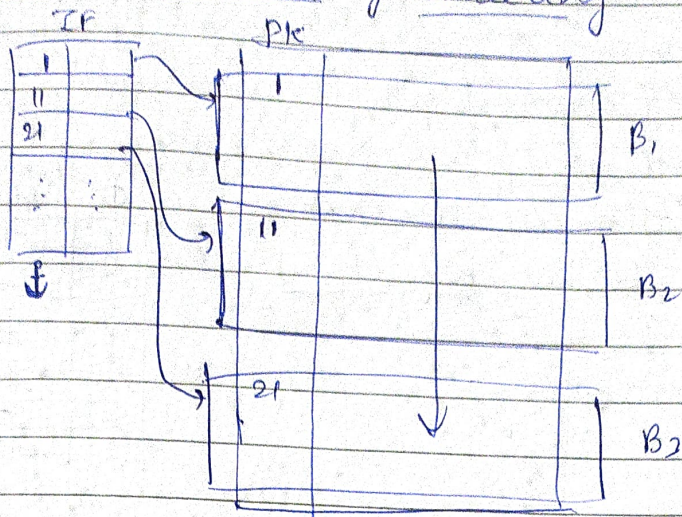
2. Multi-level Indexing



- Multiple Index files
- Use of B-Trees

Note: The Index file is always sorted irrespective of the fact whether the ^{original} file is sorted or not, in single level indexing

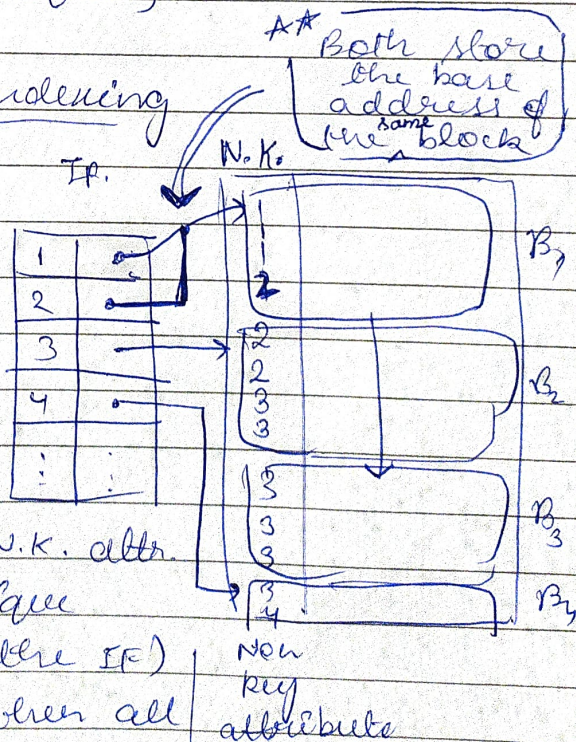
Properties of a Primary Indexing



- Data file sorted w.r.t. the primary key
- The anchor attribute in the index file is the 'primary key'
- The no. of rows in the I.F. = No. of blocks in which the data file is stored.
- Example of sparse indexing (all the records in the data file are not reflected in the index file)

Properties of Clustered Indexing

- The data file is sorted w.r.t. itsⁿ non-key attr.
- The anchor attr. is also that same non-key attr.
- The no. of rows in the IF are equal to no. of unique values in the N.K. attr.
- Dense Indexing (all unique values get a place in the IF)
- Also, sparse indexing (when all the rows in data file don't make it to the index file)



Properties of Secondary Indexing

- Main file (data file) is unsorted
- The anchor attr. and the corresp. ordering attr. in the main file can be a 'key or a non-key'
- Dense indexing

